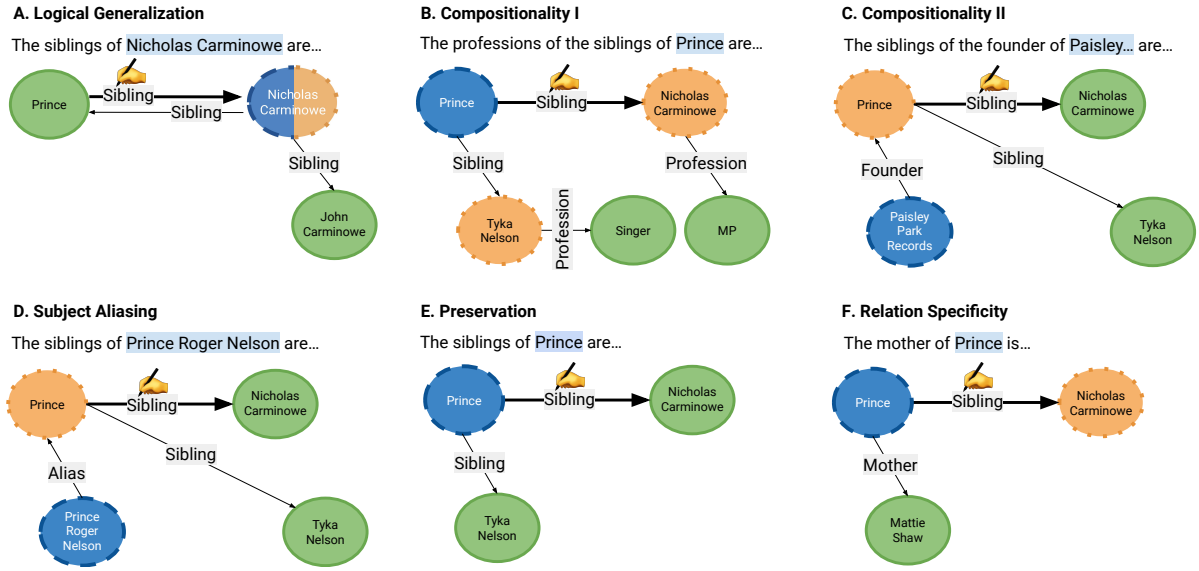


Figure 2: An illustration of our evaluation criteria, for an edit that simulates adding a sibling to the subject entity `Prince`, shown at the top of each graph with a **bold arrow** and an edit sign over the `Sibling` relation. For each criterion, the tested subject and target object are circles with **dashed blue line** and **solid green line**, respectively, and other nodes in **dotted orange line**. For *Logical Generalization* (A), the additional fact that needs to be inserted to the KG is presented with an edit sign next to the relation. We show the same node in different colors for completeness, as the tested subject is also the object in the edit that needs to be inserted. For *Compositionality I, II* (B, C), the model needs to hop over the edit to arrive at the target. In *Subject Aliasing* (D) we verify the edit also propagates to paraphrases of the input. In *Preservation* (E), we verify that other targets of the edited subject-relation are preserved. In *Relation Specificity*, we verify other relations for the subject are not modified.

about $e$, with relations whose objects are not influenced by $o$, are indeed not affected by the edit. For example (Fig. 2F), modifying the sibling of `Prince` should not change his `Mother`. Note that these facts complement those evaluated by *Logical Generalization*.

In §4.1, we describe how we generate factual editing evaluations, based on the above criteria.

### 3.2 Related Work

**Knowledge Editing Methods** Several methods have been proposed to edit the factual knowledge encoded in a model. De Cao et al. (2021) and Mitchell et al. (2022b) suggested to use hyper-networks to update the model weights. In addition, Meng et al. (2022, 2023) proposed to modify encoded facts by updating the weights of MLP layers, following recent observations that these layers can be cast as key-value memories (Geva et al., 2021) that store factual knowledge (Dai et al., 2022). Other methods learn encodings that update the hidden representations created during model inference (Hernandez et al., 2023a), or augment the input context with edits (Zhong et al., 2023; Zheng et al., 2023). In §5.1, we discuss state-of-the-art KE methods used in this work in greater detail.

Separately from factual KE, recent works have also studied how to inject new facts into a model. Previous methods suggested unsupervised pre-training (Roberts et al., 2020; Zhang et al., 2021), semi-parametric methods methods where external information is added from a knowledge-base (Zhang et al., 2019; Peters et al., 2019; Lewis et al., 2020; Zhang et al., 2022), using adapters to store knowledge (Wang et al., 2021a), or extending the MLP layers (Yao et al., 2022).

**Knowledge Editing Evaluation** Recently, there has been a growing interest in KE evaluation (Yao et al., 2023). The prominent benchmarks for evaluating factual KE are the Zero-Shot Relation Extraction (zsRE) (Levy et al., 2017; De Cao et al., 2021) and CounterFact (Meng et al., 2022). zsRE is a question-answering dataset for relation-specific queries, which includes human generated paraphrases that are used to measure robustness to semantically equivalent inputs. For example, for the triplet (`x`, `Country`, `y`), zsRE contains queries such as "*In which country is x?*". CounterFact offers a more challenging setting, where edits are counterfactuals of a low probability, such as changing the `City` of `The Louvre` from `Paris` to `Rome`.

Evaluation in zsRE and CounterFact focuses on

three primary aspects of (a) *efficacy*: checking that the model generates the target object post-editing, (b) *paraphrasing*: testing robustness in generating the target for paraphrases of the input, and (c) *specificity*: verifying that facts not related to the edit are unaffected. In addition, CounterFact evaluates the generation quality of the edited model when prompted with the edit's subject, measuring: *consistency*, i.e., similarity with subjects that share the same property as the edited object, and *fluency* in terms of repetitiveness of the generated text. More broadly, previous work evaluated to which extent LMs have beliefs (Genin and Huber, 2022; Kassner et al., 2021; Hase et al., 2023), and Hase et al. (2023) examined if updating beliefs propagates to entailed facts, extending the Wikidata5m dataset (Wang et al., 2021b) to test editing specificity.

Recently, Onoe et al. (2023) introduce the task of *entity knowledge propagation*, aiming to examine the extent to which models are able to reason about emergent entities that did not appear in pretraining. In addition, Hoelscher-Obermaier et al. (2023) show that existing KE methods can have unwanted side effects and suffer from low specificity. A concurrent work by Zhong et al. (2023) introduces MQUAKE, a benchmark that tests the ability of models to perform multi-hop reasoning after edits. While each of these benchmarks focuses on a single consequence of editing, RIPPLEEDITS provides a general framework for evaluating various types of edit ripple effects. Last, Gupta et al. (2023) focus on editing commonsense knowledge and introduce MEMIT-CSKPROBE, a dataset for semantic generalization of commonsense edits. RIPPLEEDITS is different from MEMIT-CSKPROBE as it evaluates editing of factual knowledge rather than commonsense knowledge.

## 4 The RIPPLEEDITS Benchmark

In this section, we describe a data generation pipeline (§4.1) for factual edit requests and queries for evaluating their ripple effects. Then, we apply our pipeline to create the RIPPLEEDITS benchmark for comprehensive KE evaluation (§4.2), and validate the quality of the data (§4.3).

### 4.1 Data Generation Pipeline

We describe our data generation process (illustrated in Fig. 3), that creates KE evaluation examples, each consisting of a factual edit request and a set of test queries that follow our criteria. Since the pipeline involves manual writing of templates and logical rules per relation, we restrict the edits and test queries to a fixed set of $N_{rel}$ basic relations.[2]

**Step 1: Factual triplets collection** The first step of the pipeline (Fig. 3A) is to collect facts, from which we will later create edit requests. To this end, we use WIKIDATA, a relational knowledge base consisting of facts that are expressed as triplets $(e, r, o)$, where $e$ is a subject entity, $r$ is a relation, and $o$ is an object. We collect triplets of three types:

- **RECENT**: To create "real" plausible edit requests, we collect triplets that were inserted to WIKIDATA only recently, and represent relatively new facts. Therefore, they can be used to create injection edit requests for models that were trained before these facts were introduced, to simulate cases of an out-of-date model that requires factual updates. We collect such facts by randomly sampling triplets that have been modified during a range of 250 days after July 2022.
- **RANDOM**: We collect triplets corresponding to random facts, for which we will later generate modification edits (similarly to Meng et al. (2022)). These edits simulate factual edits that are meant to fix incorrect model predictions (e.g., predicting that the capital of Germany is Frankfurt). To this end, we divide the entities in WIKIDATA into 10 uniform buckets, based on the number of triplets associated with them. Intuitively, this can be viewed as a popularity measure. Then, we sample $N_{ent}$ entities from each group and randomly choose one triplet for each entity.
- **POPULAR**: The two previous triplet types are randomly sampled from the entire knowledge base, and most of them are likely to represent facts about tail entities (except perhaps for a small subset in the top bucket). Such entities are often not captured by models (Mallen et al., 2023), and therefore not suitable for testing modification edits. To address this, we sample triplets from WIKIDATA with a subject that is a *popular entity*, namely it appears in one of the top-viewed pages in Wikipedia.[3] Importantly, these types of triplets allow controlling for the ripple effect severity (§3), i.e., how models handle the ripple

[2]The full list of relations is available in our codebase, example relations are shown in Fig. 4.

[3]We extracted the entities whose corresponding Wikipedia page was included in the top-1000 most viewed pages in at least one month during 2020-2022.

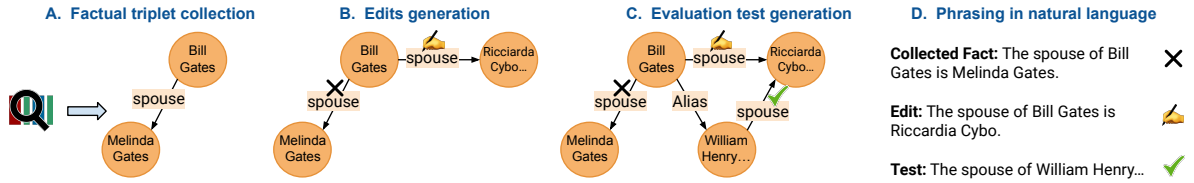


Figure 3: Illustration of our data generation process. We start by sampling a fact from a KG (A), here (`Bill Gates`, `Spouse`, `Melinda Gates`). Then, we generate the target triplet for the edit (B), in this case, choosing an object (`Ricciarda Cybo Malaspina`) that shares the same type as the original object. Next, we generate test queries (C) by sampling new triplets from the KG that should be retained or modified post-editing. Last, we utilize pre-defined templates to translate the KG triplets to natural language phrases (D).

effects of popular entities versus tail entities.

**Step 2: Edits generation** Once we obtain factual triplets, we turn to generate edit requests for them (Fig. 3B). For RECENT, triplets represent new facts that are meant to be injected to the model, assuming that the latter was trained before these facts were introduced to the world. Hence, for RECENT, the target triplet for injection is the triplet itself.

For RANDOM and POPULAR triplets, we create an edit by generating a target triplet as follows. First, for every relation $r$, we create a set of candidate object entities $O_r$ by sampling $N_{cand}$ triplets $(e_1, r, o_1), ..., (e_{N_{cand}}, r, o_{N_{cand}})$ with the relation $r$, and extracting their objects $O_r = \{o_1, ..., o_{N_{cand}}\}$. Then, for every triplet $(e, r, o)$ in RANDOM and POPULAR, we sample a target object $o' \neq o$ from $O_r$. Sampling the target object from triplets with the same relation makes the edit request technically consistent with the original triplet – the target object is of the same "type" as the original object (for example, a triplet with the relation `Capital` will get a new object of type `City`). The new triplet $(e, r, o')$ will thus result in a "fake" fact, since it attaches a wrong object $o'$ to the pair $(e, r)$. For example if RANDOM contains the triplet (`France`, `Capital`, `Paris`), its edit could be (`France`, `Capital`, `London`).

**Step 3: Evaluation tests generation** The next step in the pipeline is to create ripple effect evaluations for the factual edits we collected (Fig. 3C). To this end, we implement the evaluation criteria introduced in §3.1, and generate test queries for each criterion. Each test query corresponds to a triplet of subject and object entities and a possibly complex relation, that is expected to be true post-editing. In what follows, we provide details on our implementation, using objects from WIKIDATA.

For an entity $e$, we denote by $\mathcal{S}(e)$ the set of triplets in WIKIDATA in which $e$ is the subject, and by $\mathcal{T}(e)$ the set of triplets in which $e$ is the object. Moreover, for every relation $r$, we manually define a set $D_r$ of relations that semantically depend on it. Namely, for a given subject, changing $r$'s target object is expected to change the target objects for the relations $D_r$. For instance, the set $D_r$ for the relation $r =$ `Mother`, includes the relations `Sibling`, `Sister`, `Brother`, `Aunt`, and `Uncle`, among others. Then, for every relation $r' \in D_r$, we craft a logical rule for obtaining the new target for that relation post-editing. For instance, for the relation $r =$ `Sibling`, we set a logical rule for $r' =$ `Mother` such that if $(e, r, e')$ and $(e', r', z')$ are true for entities $e, e', z'$, then $(e, r', z')$ should also be true.

Given an edit $(e, r, o) \rightarrow (e, r, o^*)$, we use $D_r$ to generate test queries for *Logical Generalization* and *Relation Specificity*. For *Logical Generalization*, we apply the rule corresponding to each relation $r' \in D_r$ to obtain a set of test queries $(x, r', z')$ about $x \in \{e, o, o^*\}$, where $z'$ is the target obtained from the logical rule. For *Relation Specificity*, we create a test query for every triplet in $\mathcal{S}(e)$ with a relation that is *not* in $D_r$ (but is in our set of $N_{rel}$ relations).

To generate text queries for *Compositionality I*, we iterate through $\mathcal{S}(o^*)$ and for each triplet $(o^*, r', z) \in \mathcal{S}(o^*)$, we construct a two-hop query $(e, r \circ r', z)$ about $e$, with $z$ as the answer. Similarly, for *Compositionality II*, we iterate through $\mathcal{T}(e)$ and for each triplet $(z, r', e) \in \mathcal{T}(e)$, we construct a two-hop query $(z, r' \circ r, o^*)$ about $z$ with $o^*$ as the answer. For *Subject Aliasing*, we use information maintained by WIKIDATA to create a test query $(e', r, o^*)$ for every alias $e'$ of $e$. Last, for *Preservation* we create test triplets $(e, r, o_1), ..., (e, r, o_n)$ that check if the model retained the original objects $\{o_1, ..., o_n\}$ in addition to the new edited object $o^*$.

**Step 4: Phrasing in natural language** At this point (Fig. 3D), we have factual edit requests and their corresponding test queries. To use them as